

SpeedyJob 简历分析功能改进说明

一、改进的功能

改进目标为 SpeedyJob.cn 的 简历评分 页面。原有功能仅输出一个总分，未提供任何具体修改建议。

二、当前产品的问题

- 反馈缺失：只有分数，没有“哪里不好”“怎么改”的方向性指引。
- 应届生痛点：缺乏职场经验的求职者面对分数无从下手，不知道哪些经历需要突出、哪些技能需要补充，简历修改变成盲目试错。
- 行动断层：用户知道“需要优化”，但不知道“下一步具体做什么”。

三、改进方案

新增「简历优化建议」功能，支持两种输入方式：

- 粘贴简历文本
- 拖拽上传 Word 文档（.docx），自动提取文本

系统返回至少 5 条结构化建议，每条包含：

- | 类别：技能描述 / 项目经验 / 工作经历 / 教育背景 / 关键词优化等
- | 优先级：高 / 中 / 低
- | 原文片段：简历中待优化的原始文本
- | 建议文本：可直接替换的优化版本

设计理由：

1. 效率优先：面试准备时间紧张，用户需要“拿来就能改”的具体指引。
2. 精准定位：结构化输出避免泛泛而谈，每条建议都有明确修改方向。
3. 降低门槛：拖拽上传 Word 符合文件管理直觉，减少操作步骤。

技术栈：前端 Vue 3 + Vite + TailwindCSS，后端 Java 17 + Spring Boot 3.2（AI 接口预留，当前使用 Mock 实现）。

四、Vibe Coding 实践过程

工具

- 前端：VS Code + AI 编程插件 (TRAE)
- 后端：IntelliJ IDEA + AI 编程插件(TRAE,Lingma)
- 辅助构思：deepseek

工作流程（全程零手写代码）

1. 人工架构规划：deepseek 辅助设计技术选型、目录结构、核心数据流。

2. 逐文件生成：将规划拆成自然语言指令，分发给前后端 AI 逐个生成代码文件。
3. 错误迭代修复：运行报错时直接将错误日志粘贴给 AI，AI 自动修复后继续验证。
4. 功能扩展：新增「拖拽上传 Word」时，用一段自然语言描述需求，AI 完成所有代码修改。

关键 Prompt 示例

> “我要实现一个简历优化建议系统的后端，技术栈是 Java 17 + Spring Boot 3.2 ... 请按顺序生成每个文件的完整代码，确保可以直接 `mvn spring-boot:run` 启动。”

> “生成 `ResumeInput.vue` 组件，包含一个大 `textarea`，支持拖拽上传 `.docx` 并用 `mammoth` 提取文本... 请生成修改后的完整代码。”

> “运行后端报错：... 请修改对应文件并给出修复后的完整代码。”

验证方式

- 后端验证：使用 Postman 调用 `POST /api/resume/analyze`，确认返回 JSON 包含至少 5 条结构完整的建议。
- 前端验证：
 1. 粘贴纯文本简历，点击分析，确认结构化建议卡片正确渲染。
 2. 拖拽 Word 文档，确认文本自动提取并填充到输入框。
 3. 检查每条建议是否包含类别标签、优先级标识、原文与建议文本对比。
